

| 2026-03-04

| Задачи классификации и кластеризации

| Обзор класса задач

Оба класса задач подразумевают, что есть набор некоторых объектов (в широком смысле), которые обладают некоторым набором признаков (общим для всех объектов). На основании того, какие характеристики у этих объектов есть и в какой мере они выражены, мы эти объекты хотим сгруппировать.

Классификация предполагает, что группы уже выделены заранее: для каждого объекта в обучающей выборке уже есть свой класс (т.е. для каждого класса всегда есть заранее подготовленные примеры).

Классификация относится к задачам на обучение с учителем (supervised learning) и с точки зрения машинного обучения сводится к обучению модели признакам, характеризующим некоторый класс, и обобщению наборов признаков отдельных объектов класса для характеристики класса в целом.

Далее обученная модель относит любые новые данные и встреченные в них новые объекты (не входившие в обучающую выборку) к тому или иному классу, сличая их свойства с обобщенным внутренним представлением свойств (признаков) каждого класса.



Кластеризация относится к методам обучения без учителя (unsupervised learning) и ставит своей целью выяснение того, какие группы можно выделить в изначально несгруппированном наборе данных. Признаки, по которым производится разделение на группы, также выявляются в ходе обучения модели.

Модель должна разбить входные данные на N групп, при этом само число групп чаще всего задается заранее (хотя и не всегда — например, алгоритм affinity propagation сам итеративно подбирает число кластеров, пока не стабилизируется). Необходимое число групп можно приблизительно определить в ходе предварительного анализа данных, используя, например, ранее рассмотренный метод локтя.

Note

Применительно к кластерному анализу это часто также подразумевает итеративное выполнение алгоритма с разным целевым числом кластеров и сравнением метрик качества кластеризации для разного числа кластеров.

| Классификация

Классификация — задача обучения с учителем: у объектов есть признаки, а классы известны заранее по размеченной выборке.

Модель учится обобщать признаки классов на обучающих данных и затем относит новые объекты к классам по этому внутреннему представлению

| Подготовка данных

Базовое правило:

1. Разделить данные на train/test (часто 80%/20%) и не трогать test до финальной оценки.
2. При подборе гиперпараметров использовать кросс-валидацию (**StratifiedKfold** для классификации), чтобы не «подсматривать» в test.

На этом этапе можно сделать критическую ошибку — допустить **утечку данных (data leakage)**: когда статистики предобработки (масштабирование, PCA) вычисляются на всех данных, а потом применяется train/test split. Это завышает метрики и ломает смысл эксперимента.

| Метрики качества классификации

1. **Accuracy** (доля верных ответов) — полезна, но может быть обманчива на несбалансированных данных (редкий «положительный» класс).
2. **Precision/Recall** —
 1. **Precision (точность)** — доля **правильных положительных предсказаний** среди всех предсказанных положительных случаев
$$\text{Precision} = \frac{TP}{TP+FP}$$

2. **Recall (полнота)** — доля **настоящих положительных случаев**, которые модель правильно идентифицировала $\text{Recall} = \frac{TP}{TP+FN}$

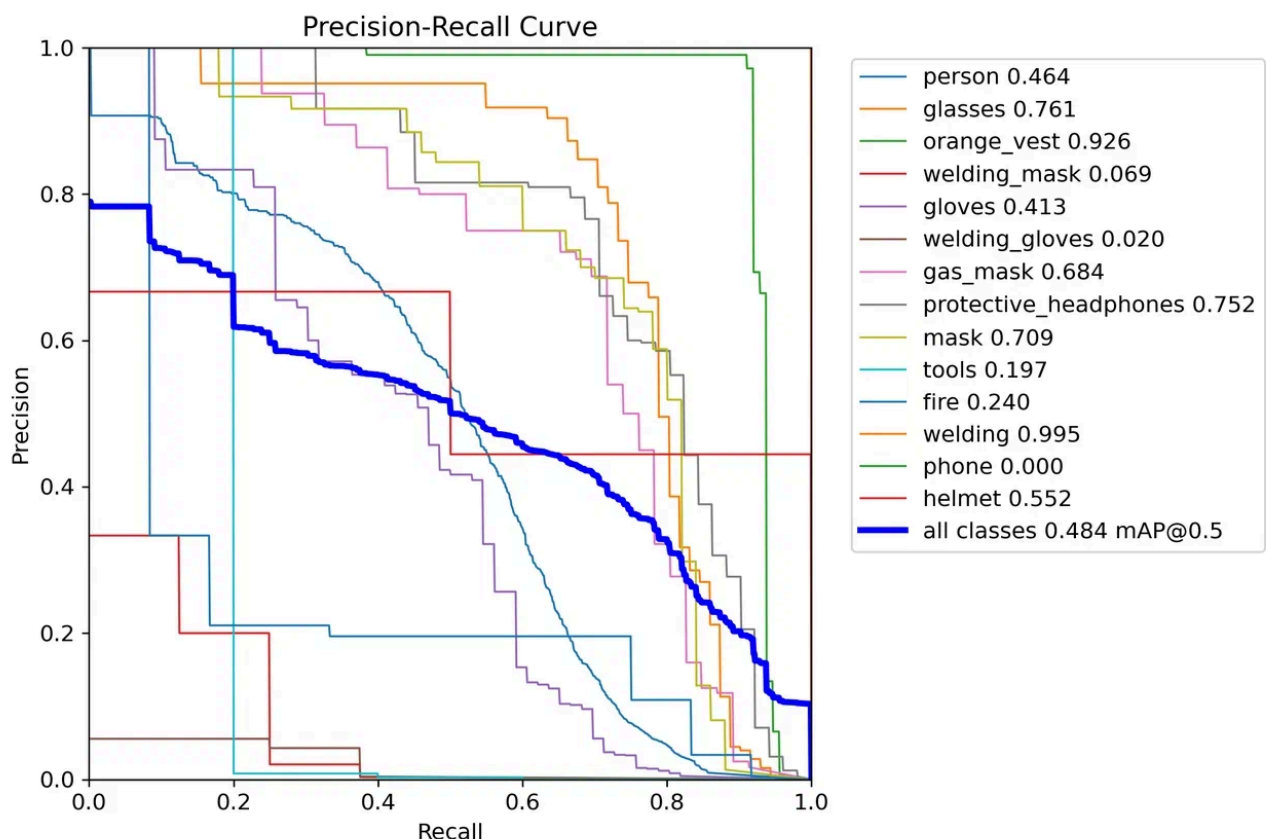
3. **F1-score** — **гармоническое среднее** между **точностью (precision)** и **полнотой (recall)**.

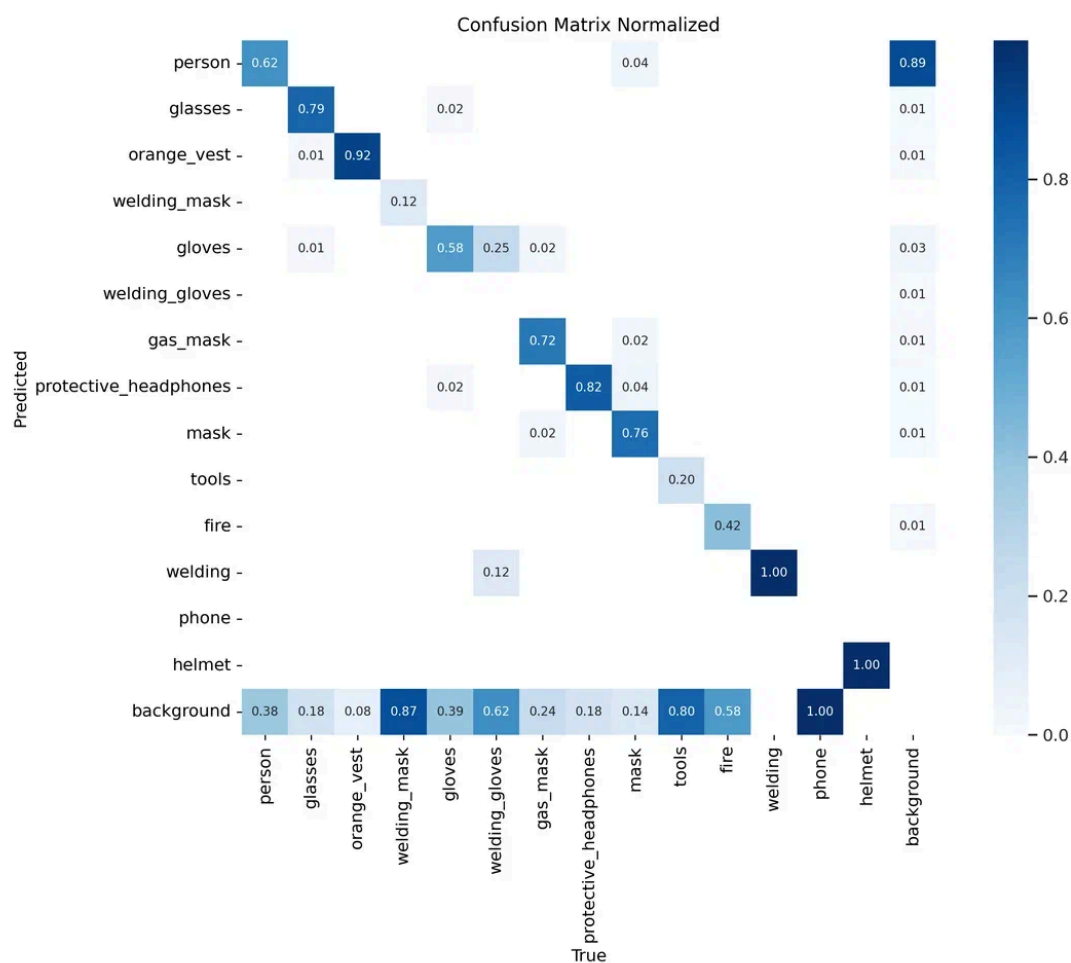
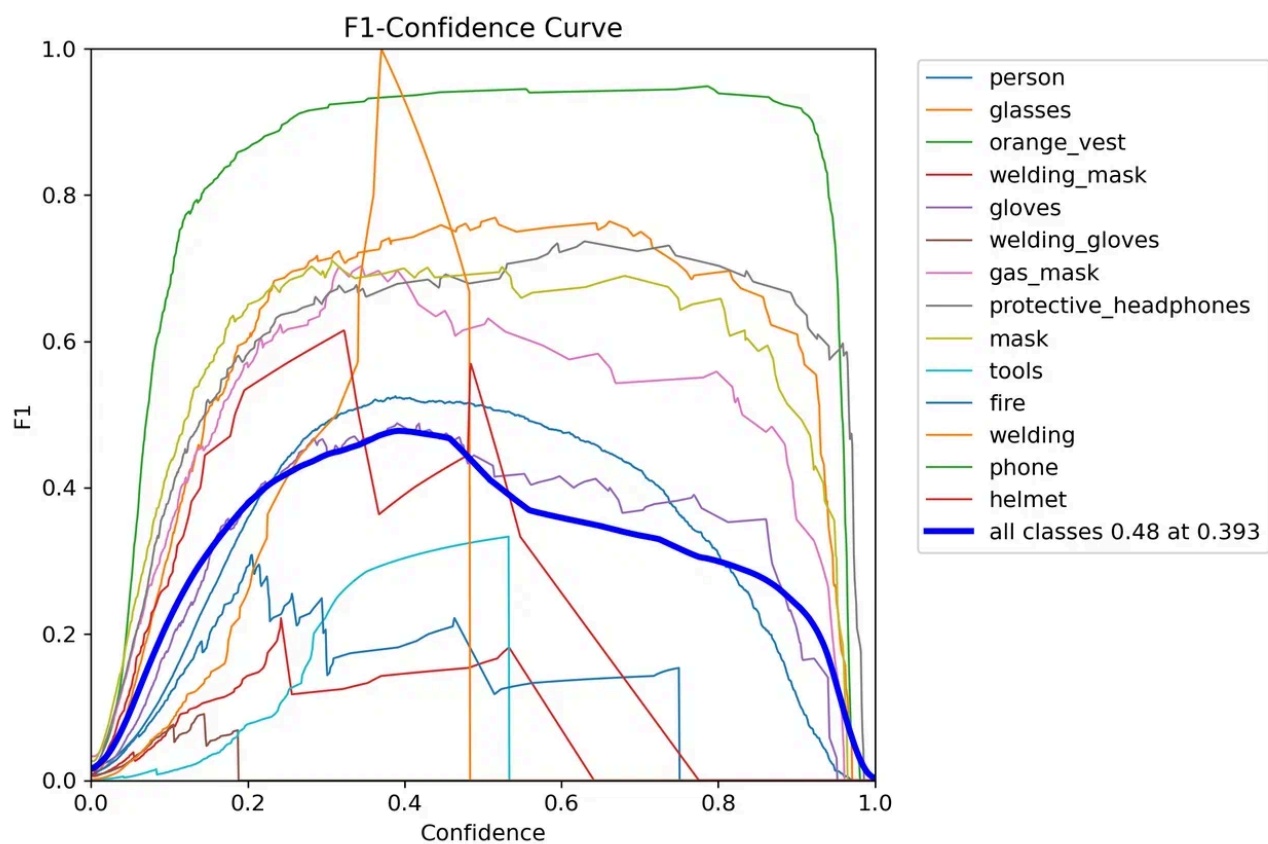
$$F1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{2 \times TP}{2 \times TP + FP + FN},$$

TP — истинно положительные, **FP** — ложноположительные, **FN** — ложноотрицательные результаты.

4. **Confusion matrix** — матрица ошибок, сравнивает **фактические метки** (истинные значения) с **предсказанными метками** (результатами модели)

5. **ROC-AUC** или **PR-AUC** (по ситуации) — **ROC-AUC** оценивает способность модели **ранжировать** объекты: вероятность того, что случайно выбранный положительный объект (правильный класс) будет классифицирован с более высокой вероятностью, чем случайно выбранный отрицательный (неправильный класс); **Precision-Recall AUC** фокусируется **исключительно на положительном классе**, измеряя баланс между **точностью (precision)** и **полнотой (recall)**. Она **более устойчива к сильному дисбалансу**, особенно когда положительный класс редкий (например, детекция мошенничества, редкие болезни). В таких случаях ROC-AUC может быть оптимистично завышенной, а PR-AUC дает более реалистичную оценку.





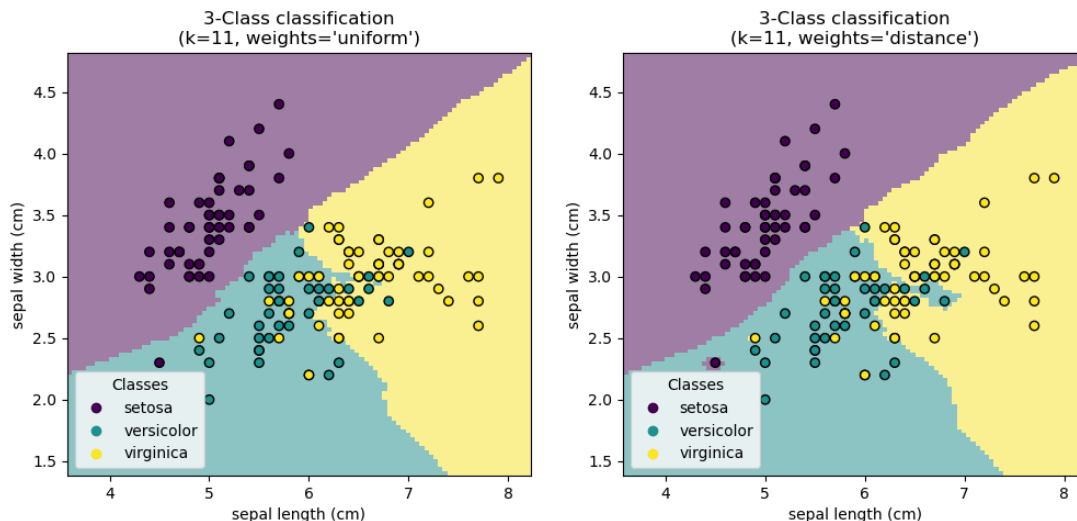
Обзор базовых алгоритмов

к-ближайших соседей (kNN)

kNN классифицирует объект по голосованию среди k ближайших объектов в пространстве признаков.

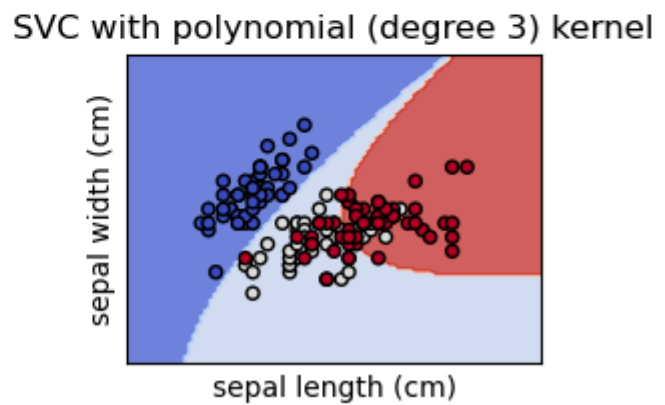
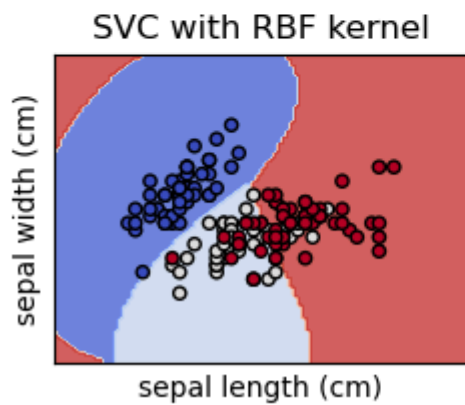
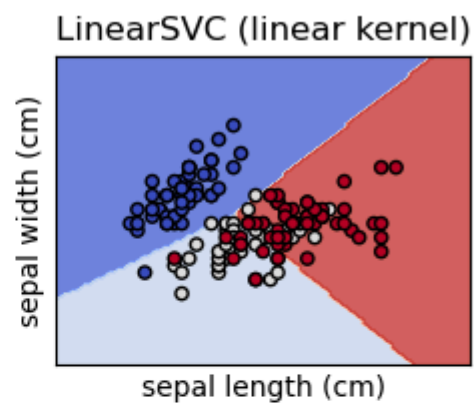
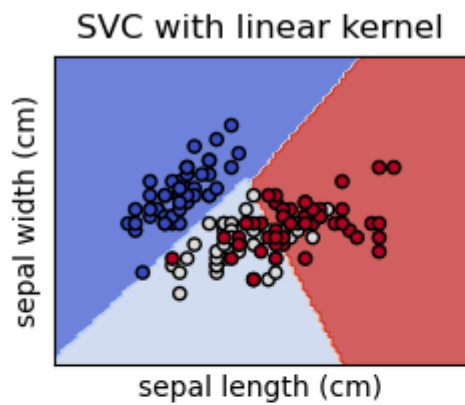
Плюсы: простота, понятная интерпретация «похожести».

Минусы: чувствителен к масштабу признаков и к «проклятию размерности», поэтому часто требует нормализации и иногда PCA.



I SVM (support vector machines, машины опорных векторов)

SVM строит разделяющую границу (гиперплоскость) — в нелинейных случаях используется разные ядра (kernel trick). На небольших/средних датасетах часто показывает себя очень хорошо, но требует аккуратного подбора гиперпараметров и обычно чувствителен к масштабированию.



I Деревья решений (Decision Tree) и случайный лес (Random Forest)

Дерево решений хорошо объяснимо, но склонно к переобучению без ограничений глубины/листьев.

Decision tree trained on all the iris features



Случайный лес — ансамбль деревьев, который обычно повышает устойчивость и качество по сравнению с одним деревом.

Градиентный бустинг по деревьям (GBDT)

Идея бустинга: строим сильную модель как сумму слабых моделей (обычно небольших деревьев), где каждый следующий шаг учится исправлять ошибки текущего ансамбля.

В градиентном бустинге «ошибка» формализуется через функцию потерь: на каждом шаге новая модель приближает отрицательный градиент (pseudo-response) и добавляется в ансамбль, улучшая качество.

Для табличных данных (производственные параметры, телеметрия, финансовые признаки) GBDT часто дает отличное качество без тяжелых нейросетей и является стандартным выбором в индустрии; это типичное решение для табличных данных (конкретно **XGBoost/LightGBM**).

Основные гиперпараметры

1. **n_estimators** : число деревьев; больше — обычно лучше (до переобучения).
2. **learning_rate** (shrinkage): чем меньше, тем более «осторожно» учимся, но обычно нужно больше деревьев.

3. `max_depth` / `max_leaf_nodes` / `min_samples_leaf`: управляют сложностью базовых деревьев и риском переобучения.
4. `subsample` / `colsample_*` (в некоторых реализациях): стохастика, помогает регуляризовать.

В продакшн-средах используют XGBoost/LightGBM/CatBoost в зависимости от ограничений (скорость обучения, размер данных, наличие категориальных признаков, требования к интерпретируемости и инфраструктуре)

Мы будем смотреть на реализацию `sklearn`
(`GradientBoostingClassifier` / `HistGradientBoostingClassifier`)

| Кластеризация

Кластеризация относится к обучению без учителя: классы заранее не известны, алгоритм сам пытается найти группы в данных.

Число кластеров часто задают заранее и подбирают по метрикам качества, в том числе через «метод локтя» (аналогично идее из PCA) и через **коэффициент силуэта**.

[Коэффициент силуэта](#) — мера пересечения кластеров друг с другом. Коэффициент силуэта оценивает, насколько точки ближе к своему кластеру (т.е. к тому, к которому их отнес алгоритм), чем к ближайшему соседнему; его среднее значение удобно использовать для выбора числа кластеров.

Коэффициент силуэт — рассчитывается с использованием среднего расстояния внутри кластера (`a`) и среднего расстояния до ближайшего кластера (`b`) для каждого образца, т.е. насколько каждый образец похож на другие в своем кластере и непохож на образцы в другом ближайшем кластере

Идеальное значение — 1 (внутри группы все похожи, на другую группу никто не похож); худшее — -1 (кластеры подобраны неверно); 0 — кластеры перекрывают друг друга (описанная выше ситуация с объектами на границах и компромиссных решениях)

| Обзор базовых алгоритмов

| Алгоритм k -средних (иногда — алгоритм Ллойда)

k -means минимизирует внутрикластерную сумму квадратов расстояний до центроидов (инерцию), выбирая такие центры кластеров, для которых сумма квадратов расстояний от центра до точек минимальна.

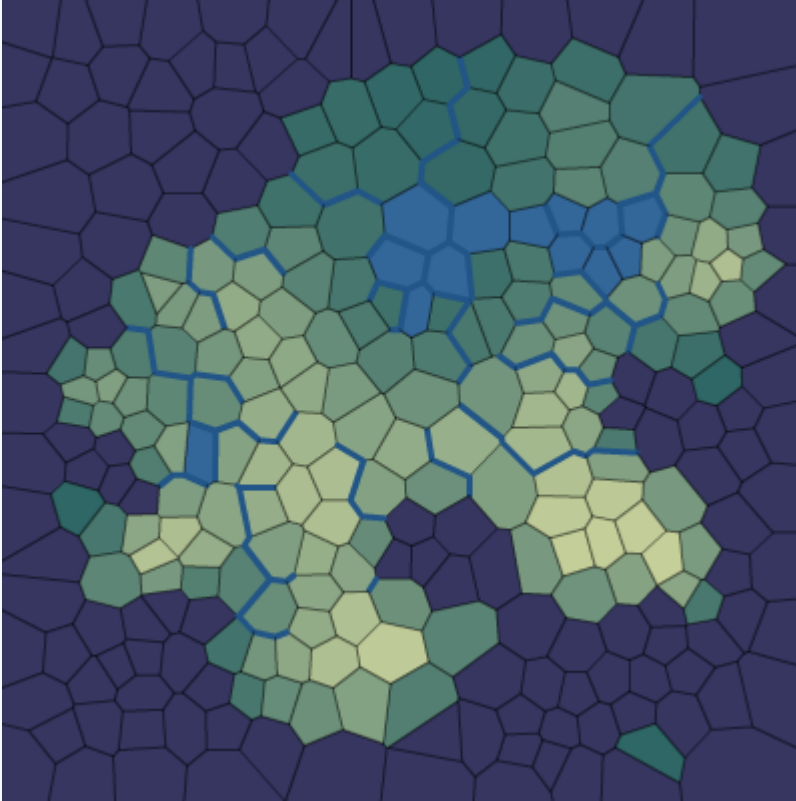
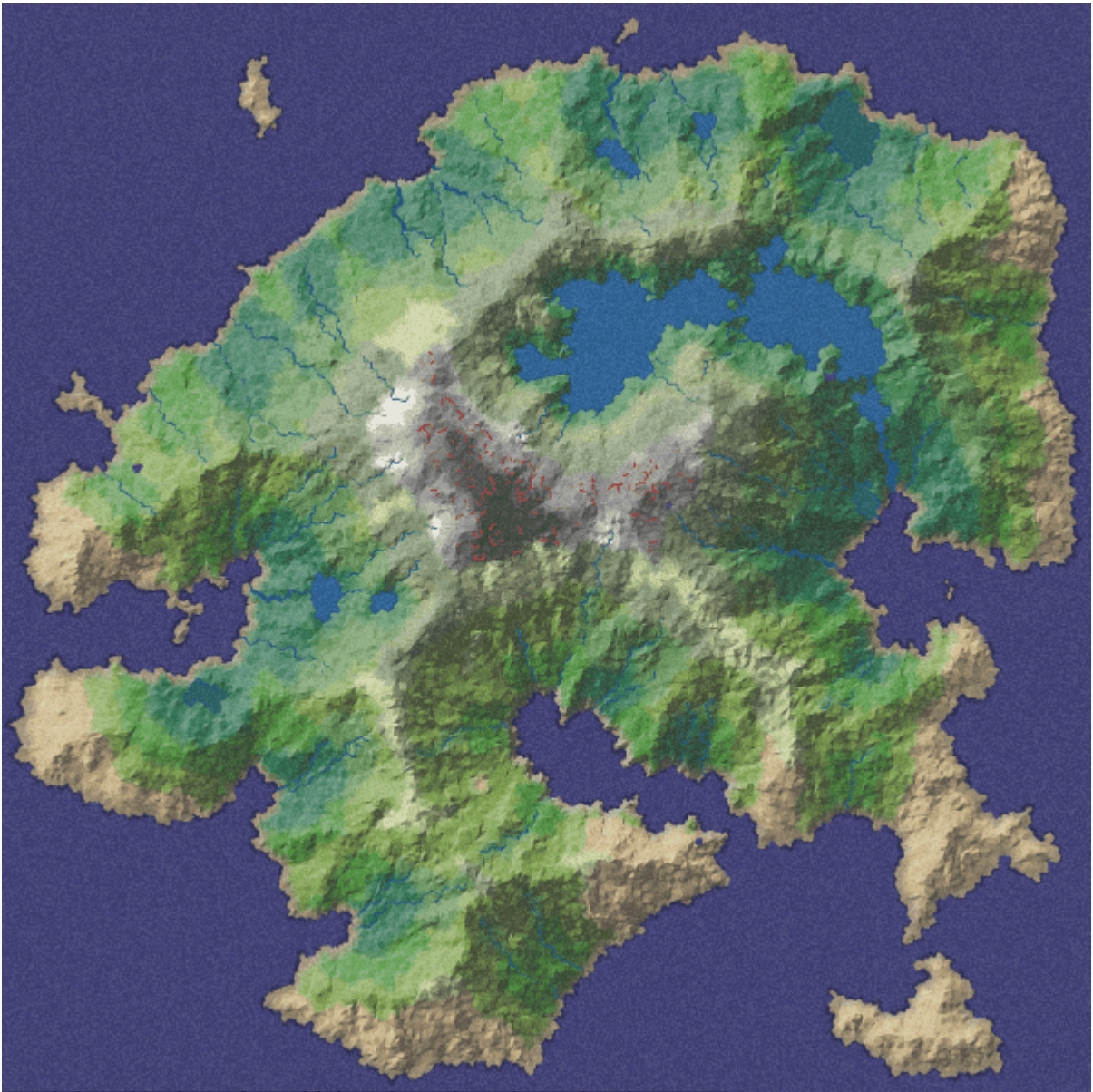
Страдает от т.н. «проклятия размерности» — т.к. инерция не является нормализованной метрикой, то Евклидовы расстояния для пространств высоких размерностей быстро возрастают. Поэтому часто метод k -средних используют в связке с методами снижения размерности (например, методом главных компонент).

Алгоритм требует задания числа кластеров, поэтому также нужно применять метод локтя или аналогичные способы поиска оптимального числа кластеров.

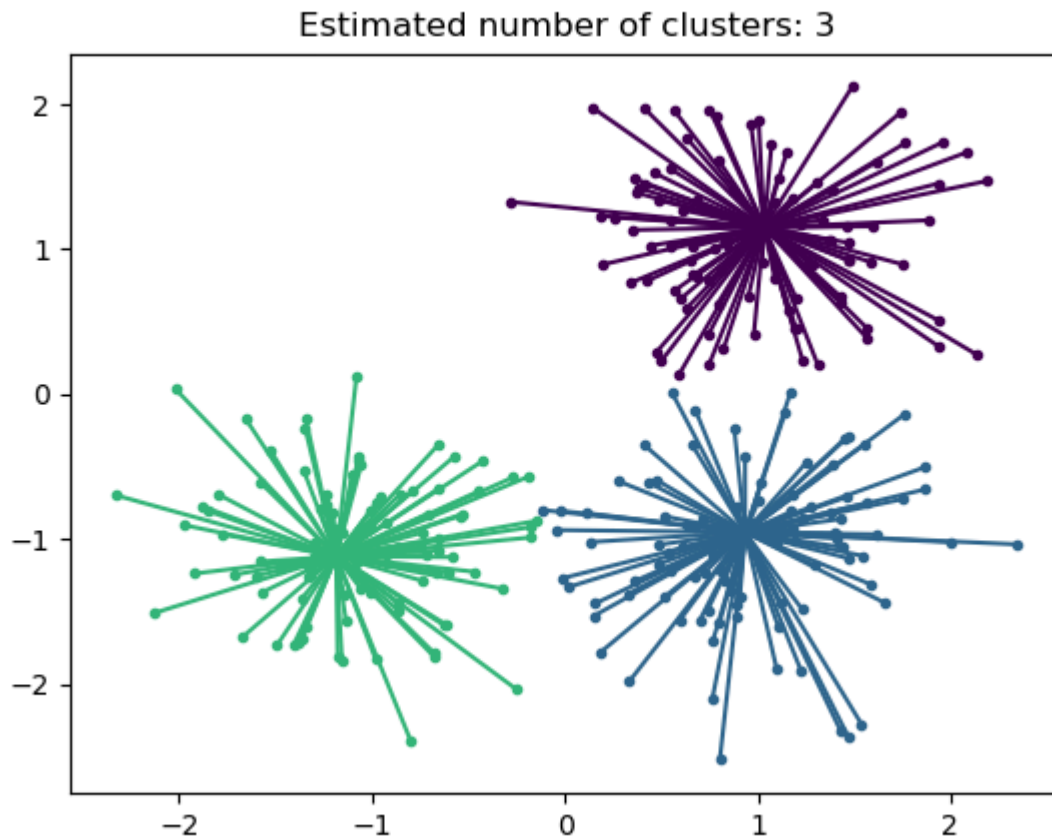
Info

Похожим на k -средние образом считаются диаграммы Вороного

<http://www-cs-students.stanford.edu/~amitp/game-programming/polygon-map-generation/>



Алгоритм распространения близости (Affinity Propagation)



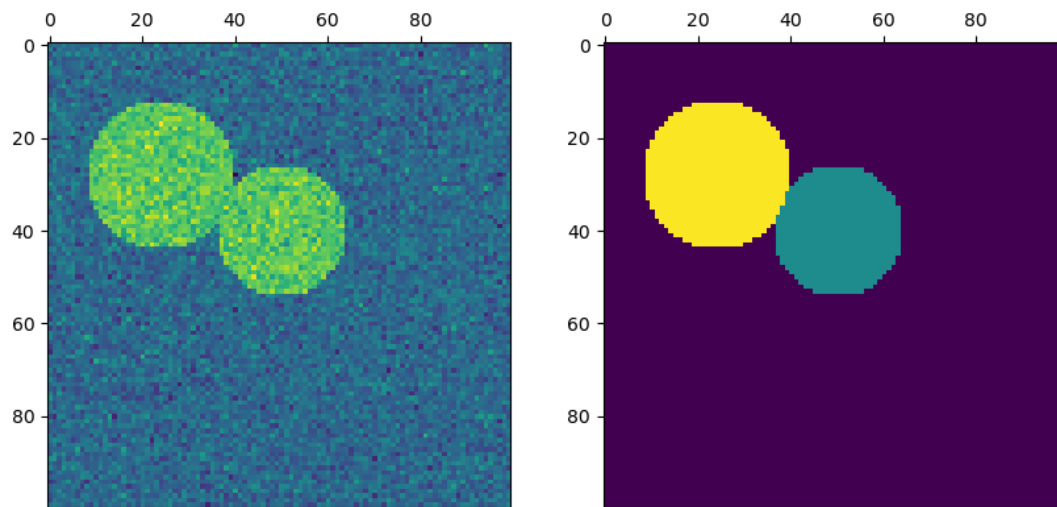
В отличие от многих других кластерных алгоритмов не требует задания числа кластеров.

Алгоритм итеративно посылает «сообщения» между парами образцов (объектов) пока не наступит эквilibrium — нельзя будет переместить образец из одной группы в другую. Сообщения несут смысл того, насколько один элемент в паре может служить образцом для другого (т.е. насколько они похожи между собой)

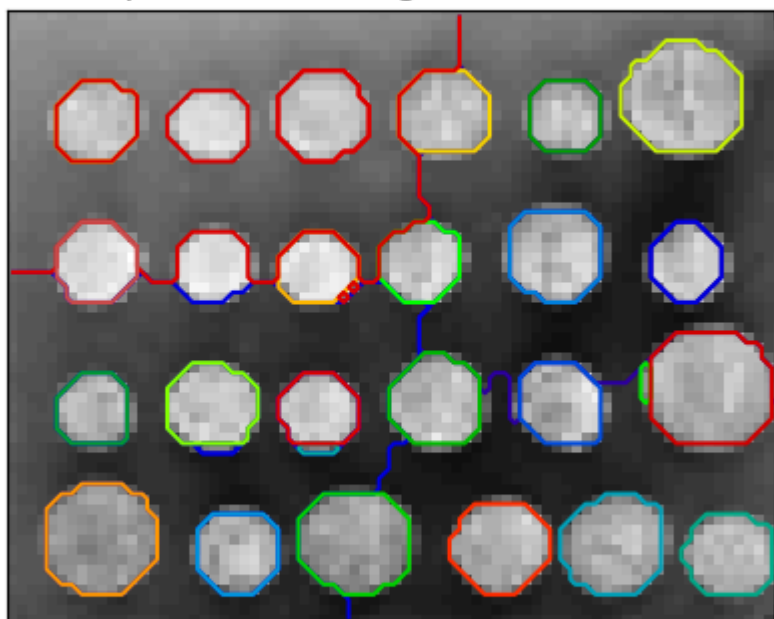
Иногда выходят компромиссные варианты (т.е. объекты на границах кластеров не имеют четко выраженной принадлежности ни к одной из двух групп и оказались в какой-либо из них просто по достижению предела и общего эквilibriumа)

Спектральная кластеризация (spectral clustering)

- работает с матрицами сходства (affinity/similarity matrices) в отличие от матриц расстояния
- производится свертка (embedding) матрицы, после чего к уже этой свертке применяется другой алгоритм кластеризации (чаще всего — k -средние) — поэтому спектральной кластеризации тоже нужно число кластеров (см. метод локтя)
- хорошо работает для разреженных матриц (sparse matrices)
- хорошо подходит для решения задачи сегментации (например, изображений)



Spectral clustering: kmeans, 1.76s



Note

«Красивые» сходства:

$$similarity = np. \exp(-beta * distance / distance. std())$$

```
delta = 2 # delta is a free parameter representing the width of the
Gaussian kernel.
```

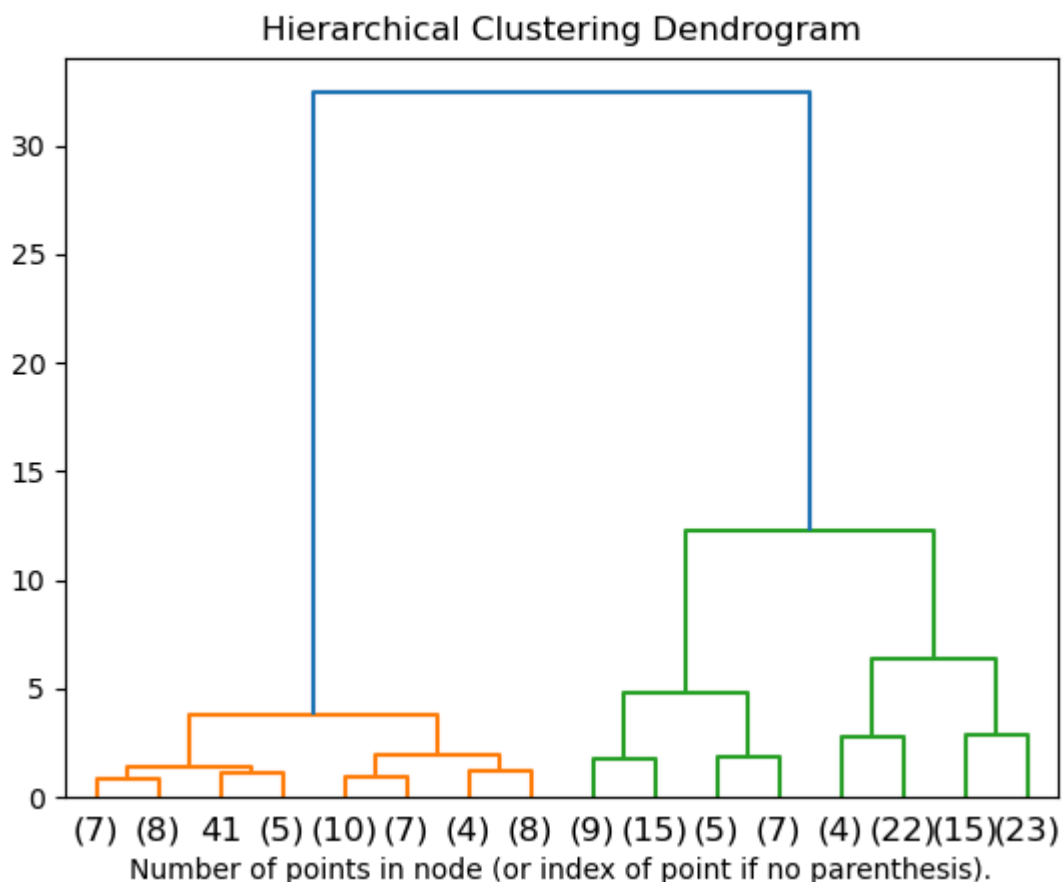
```
similarity_matrix = np.exp(- matrix ** 2 / (2. * delta ** 2))
```

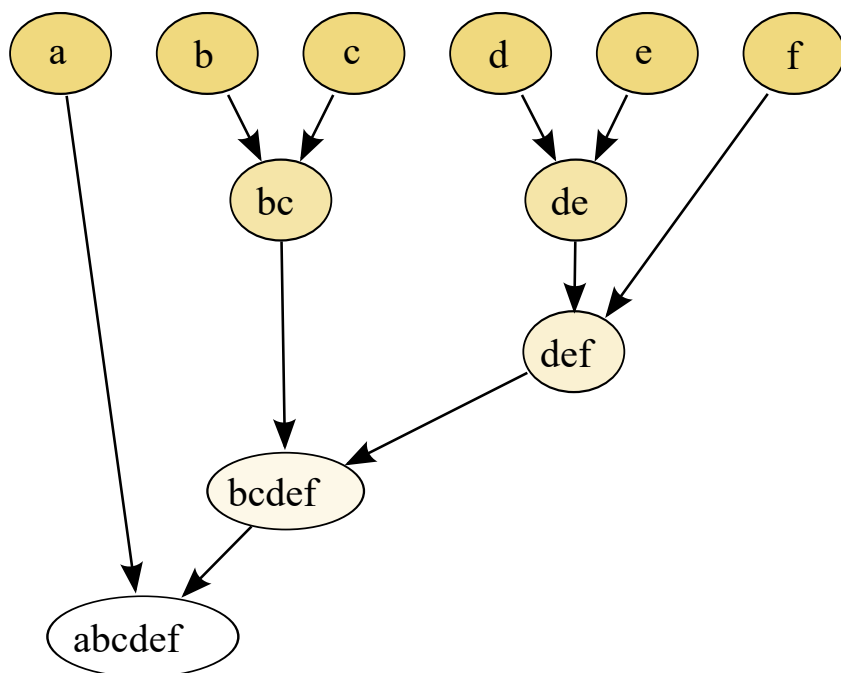
«Удобные» сходства (когда надо решить задачу быстро, но жертвуя «смыслом» меры — quick and dirty):

$$similarity = 1 - distance$$

Иерархическая кластеризация (hierarchical) / агломеративная (agglomerative)

- семейство алгоритмов, образующих иерархическую структуру из кластеров (более мелкие кластеры входят в более крупные)
- такая иерархия чаще всего визуализируется при помощи дендрограммы — древовидной диаграммы, причем «корневым элементом» такого дерева является единый общий кластер, объединяющий все элементы, тогда как ветви — кластеры, включающие в себя только некоторые элементы, а листья — непосредственно сами образцы (элементы)

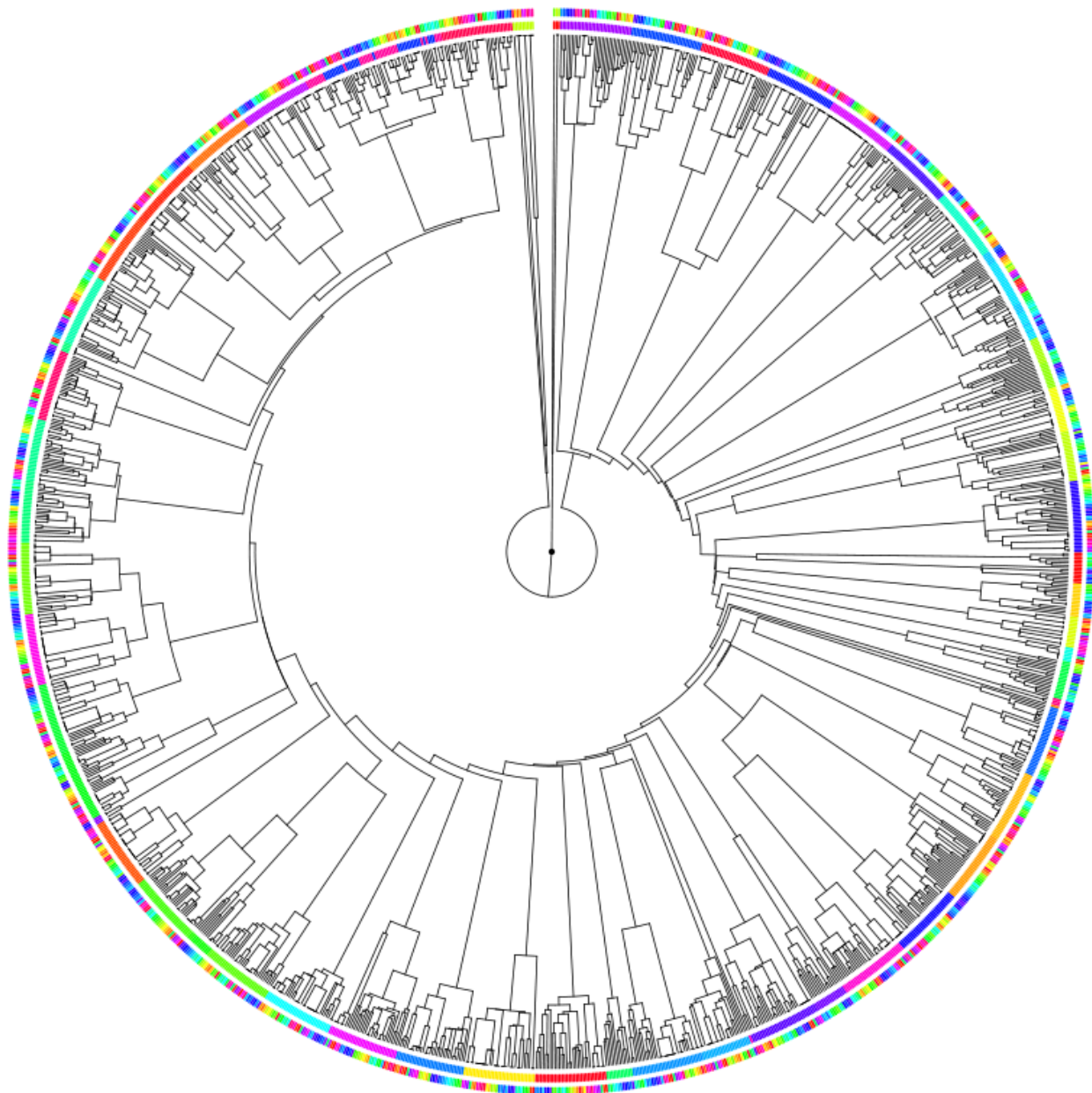




[A Visual Expedition Inside the Linux File Systems - Linux Kernel 2.6.x](#) — в примере несколько экспериментов с разными мерами расстояния (на иллюстрации — расстояние Канберры)

Info

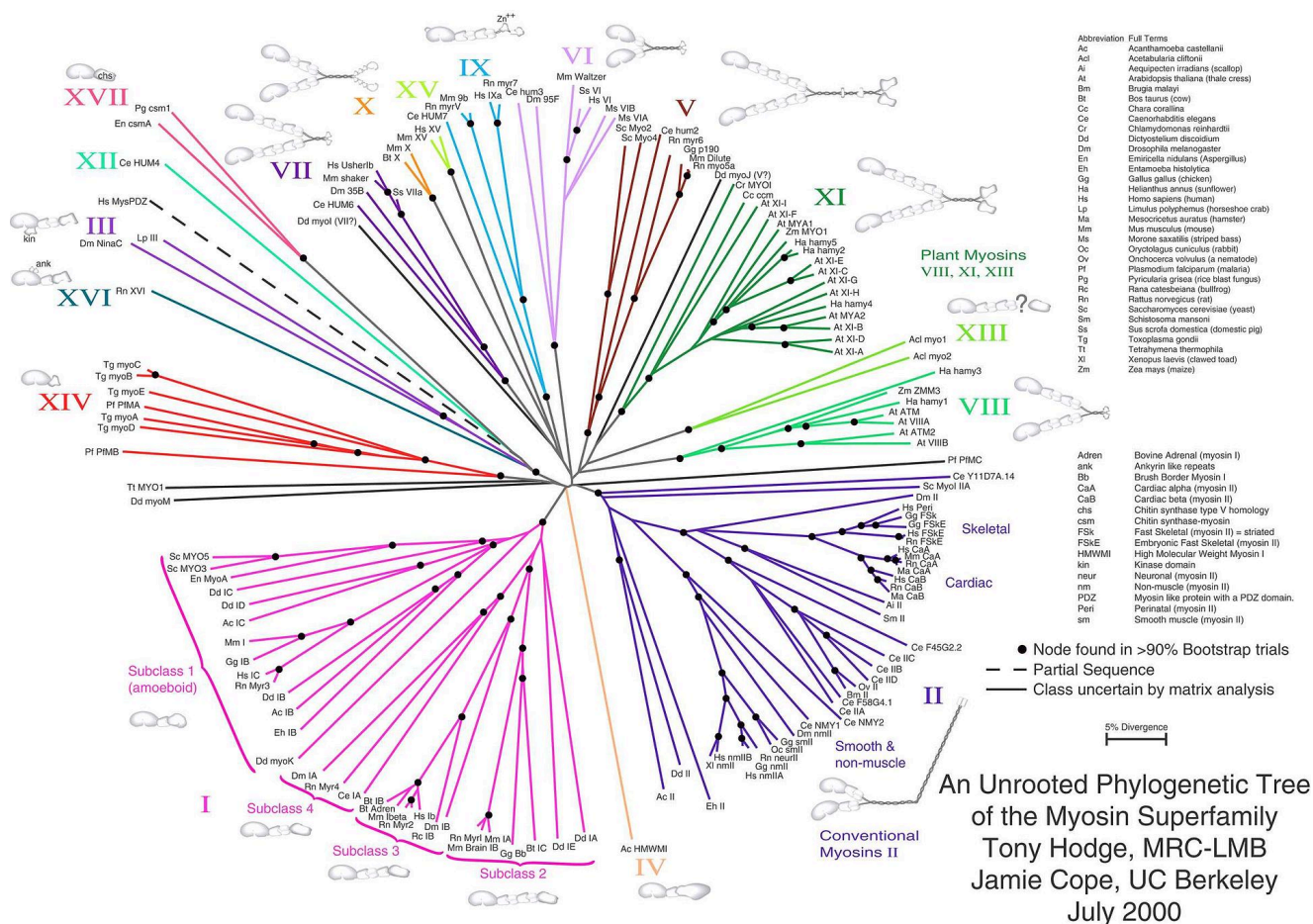
Расстояние Канберры — взвешенная версия Манхэттенского расстояния, представлена данная мера была в 1960-х годах



Агломеративная кластеризация — частный случай иерархической кластеризации, когда мы движемся «снизу вверх»: от отдельных элементов к объединяющим их кластерам.

Если мы движемся «сверху вниз», то такая иерархическая кластеризация называется **разделительной** (divisive) — мы начинаем с «корня» и рекурсивно разделяем элементы по иерархии (ступеням)

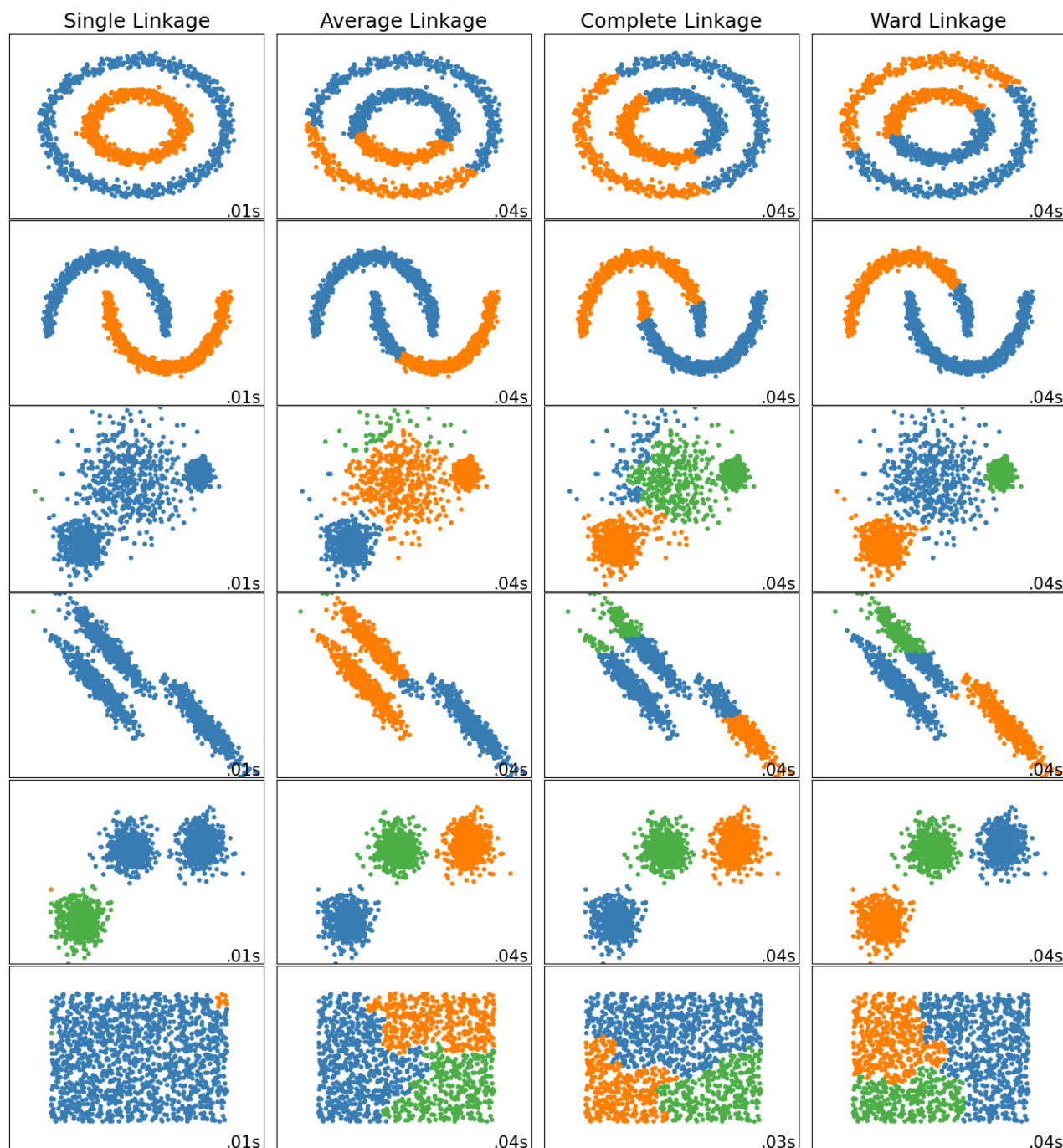
Данные алгоритмы активно применяются в биологии и связанных отраслях научного знания, например, филогенетическое древо:



Алгоритмы иерархической кластеризации различаются тем, какие формулы для расчета связи между элементами используются:

- **Метод Уорда (Минимальное увеличение суммы квадратов, MISSQ) (Ward clustering)** — В качестве расстояния между кластерами берётся прирост суммы квадратов расстояний объектов до центра кластера, получаемого в результате их объединения. На каждом шаге алгоритма объединяются такие два кластера, которые приводят к минимальному увеличению дисперсии. Этот метод применяется для задач с близко расположенными кластерами.
- **Метод полной связи (maximum/complete linkage)** — расстояние между кластерами равно максимальному расстоянию между двумя элементами из данных кластеров, «метод дальнего соседа»
- **Метод одиночной связи (single linkage)** — расстояние между кластерами равно минимальному расстоянию между двумя элементами из данных кластеров, «метод ближайшего соседа»
- **Метод средней связи (average linkage)** — расстояние между кластерами равно средней дистанции между всеми элементами этих двух кластеров, бывает взвешенный (WPGMA) и невзвешенный (UPGMA) — отличаются тем, что в одном случае связи могут иметь коэффициент веса, что увеличивает или уменьшает их влияние на итоговое расстояние между кластерами

Сравнение разных подходов к связям есть в примере в документации [scikit-learn](#):



Лабораторная работа №3

1. Загрузить из наборов данных Scikit-learn набор «Ирисы Фишера» (https://scikit-learn.org/stable/auto_examples/datasets/plot_iris_dataset.html)
2. Ознакомиться с алгоритмами классификации (https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html) и кластеризации (<https://scikit-learn.org/stable/modules/clustering.html>, https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html).
3. Провести классификацию набора алгоритмом k -ближайших соседей. Визуализировать результат.

4. Провести классификацию набора алгоритмом «случайный лес». Визуализировать результат.
5. Провести классификацию набора машинами опорных векторов (SVM). Визуализировать результат.
6. Провести кластеризацию набора алгоритмом k -средних. Визуализировать результат.
7. Провести иерархическую кластеризацию методом Уорда. Визуализировать дендрограмму.
8. Провести спектральную кластеризацию. Визуализировать результат.
9. Используя набор данных о пульсарах HTRU2 (приложенный архив или <https://archive.ics.uci.edu/ml/datasets/HTRU2>), исследовать набор данных и реализовать классифицирующую модель любым подходящим методом (но не ИНС! — Gradient Boosting (sklearn) / RandomForest / SVM).
 - рекомендуется в числе прочего использовать `pairplot` (библиотека `seaborn`) для визуального исследования датасета
 - при решении важно отметить и описать особенности набора данных
 - в ходе решения задачи нужно также ответить на вопрос, достаточна ли точность в 98% для такого набора данных
10. Используя набор публикаций о здоровье и медицине (приложенный архив или <https://archive.ics.uci.edu/ml/datasets/Health+News+in+Twitter>), исследовать набор данных и реализовать кластеризующую модель любым подходящим методом (но не ИНС!).